

Societal Impacts

How AI is transforming work at Anthropic

2. dec. 2025

How is AI changing the way we work? Our previous research on AI's economic impacts looked at the labor market as a whole, covering a variety of different jobs. But what if we studied some of the earliest adopters of AI technology in more detail—namely, us?

Turning the lens inward, in August 2025 we surveyed 132 Anthropic engineers and researchers, conducted 53 in-depth qualitative interviews, and studied internal Claude Code usage data to find out how AI use is changing things at Anthropic. We find that AI use is radically changing the nature of work for software developers, generating both hope and concern.

Our research reveals a workplace facing significant transformations: Engineers are getting a lot more done, becoming more “full-stack” (able to succeed at tasks beyond their normal expertise), accelerating their learning and iteration speed, and tackling previously-neglected tasks. This expansion in breadth also has people wondering about the trade-offs—some worry that this could mean losing deeper technical competence, or becoming less able to effectively supervise Claude's outputs, while others embrace the opportunity to think more expansively and at a higher level. Some found that more AI collaboration meant they collaborated less with colleagues; some wondered if they might eventually automate themselves out of a job.

We recognize that studying AI's impact at a company building AI means representing a privileged position—our engineers have early access to cutting-edge tools, work in a relatively stable field, and are themselves contributing to the AI transformation affecting other industries. Despite this, we felt it was on balance useful to research and publish these findings, because what's happening inside Anthropic for engineers may still be an instructive harbinger of broader societal transformation. Our findings imply some challenges and considerations that may warrant early attention across sectors (though see the Limitations

section in the [Appendix](#) for caveats). At the time this data was collected, Claude Sonnet 4 and Claude Opus 4 were the most capable models available, and capabilities have continued to advance.

More capable AI brings productivity benefits, but it also raises questions about maintaining technical expertise, preserving meaningful collaboration, and preparing for an uncertain future that may require new approaches to learning, mentorship, and career development in an AI-augmented workplace. We discuss some initial steps we're taking to explore these questions internally in the Looking Forward section below. We also explored potential policy responses in our recent blog post on [ideas for AI-related economic policy](#).

Key findings

In this section, we briefly summarize the findings from our survey, interviews, and Claude Code data. We provide detailed findings, methods, and caveats in the subsequent sections below.

Survey data

1. **Anthropic engineers and researchers use Claude most often for fixing code errors and learning about the codebase.** Debugging and code understanding are the most common uses (Figure 1).
2. **People report increasing Claude usage and productivity gains.** Employees self-report using Claude in 60% of their work and achieving a 50% productivity boost, a 2-3x increase from this time last year. This productivity looks like slightly less time per task category, but considerably more output volume (Figure 2).
3. **27% of Claude-assisted work consists of tasks that wouldn't have been done otherwise**, such as scaling projects, making nice-to-have tools (e.g. interactive data dashboards), and exploratory work that wouldn't be cost-effective if done manually.
4. **Most employees use Claude frequently while reporting they can “fully delegate” 0-20% of their work to it.** Claude is a constant collaborator but using it generally involves active supervision and validation, especially in high-stakes work—versus handing off tasks requiring no verification at all.

Qualitative interviews

1. **Employees are developing intuitions for AI delegation.** Engineers tend to delegate tasks that are easily verifiable, where they “can relatively easily sniff-check on correctness”, low-stakes (e.g. “throwaway debug or research code”), or boring (“The more excited I am to do the task, the more likely I am to not use Claude”). Many describe a trust progression, starting with simple tasks and

gradually delegating more complex work—and while they’re currently keeping most design or “taste” tasks, this boundary is being renegotiated as models improve.

2. **Skillsets are broadening into more areas, but some are getting less practice.** Claude enables people to broaden their skills into more areas (of software engineering (“I can very capably work on front-end, or transactional databases... where previously I would've been scared to touch stuff”), but some employees are also concerned, paradoxically, about the atrophy of deeper skillsets required for both writing and critiquing code—“When producing output is so easy and fast, it gets harder and harder to actually take the time to learn something.”
3. **Changing relationship to coding craft.** Some engineers embrace AI assistance and focus on outcomes (“I thought that I really enjoyed writing code, and I think instead I actually just enjoy what I get out of writing code”); others say that “there are certainly some parts of [writing code] that I miss.”
4. **Workplace social dynamics may be changing.** Claude is now the first stop for questions that used to go to colleagues—some report fewer mentorship and collaboration opportunities as a result. (“I like working with people and it's sad that I ‘need’ them less now... More junior people don't come to me with questions as often.”)
5. **Career evolution and uncertainty.** Engineers report shifting toward higher-level work managing AI systems and report significant productivity gains. However, these changes also raise questions about the long-term trajectory of software engineering as a profession. Some express conflicting feelings about the future: “I feel optimistic in the short term but in the long term I think AI will end up doing everything and make me and many others irrelevant.” Others emphasize genuine uncertainty, saying only that it was “hard to say” what their roles might look like in a few years’ time.

Claude Code usage trends

1. **Claude is handling increasingly complex tasks more autonomously.** Six months ago, Claude Code would complete about 10 actions on its own before needing human input. Now, it generally handles around 20, needing less frequent human steering to complete more complex workflows (Figure 3). Engineers increasingly use Claude for complex tasks like code design/planning (1% to 10% of usage) and implementing new features (14% to 37%) (Figure 4).
2. **Claude fixes a lot of “papercuts”.** 8.6% of Claude Code tasks involve fixing minor issues that improve quality of life, like refactoring code for maintainability (that is, “fixing papercuts”) that people say would typically be deprioritized. These small fixes could add up to larger productivity and efficiency gains.
3. **Everyone is becoming more “full-stack”.** Different teams use Claude in different ways, often to augment their core expertise—Security uses it to analyze unfamiliar code, Alignment & Safety use it to build front-end

visualizations of their data, and so on (Figure 5).

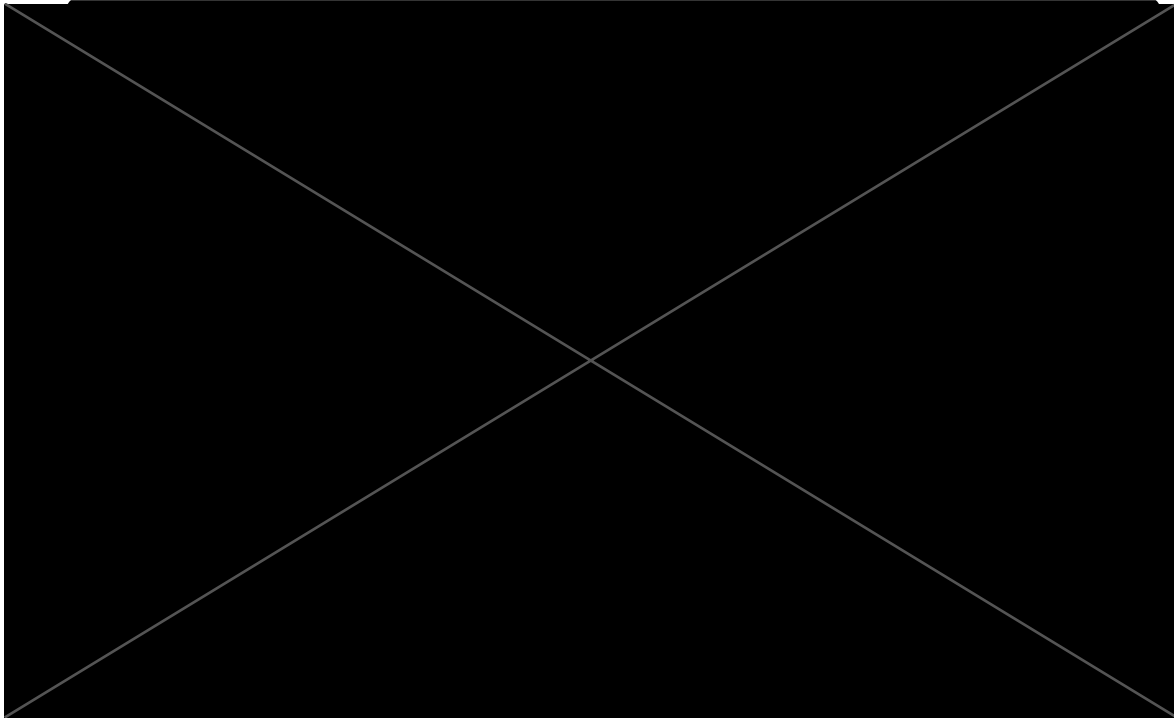
Survey data

We surveyed 132 Anthropic engineers and researchers from across the organization about their Claude use, to better understand how exactly they were using it day-to-day. We distributed our survey through internal communication channels and direct outreach to employees across diverse teams representing both research and product functions. We have included a Limitations section in the Appendix with more methodological details, and we are sharing [our survey questions](#) so others can evaluate our approach and adapt it for their own research.

What coding tasks are people using Claude for?

We asked the surveyed engineers and researchers to rate how often they used Claude for various types of coding tasks, such as “debugging” (using Claude to help fix errors in code), “code understanding” (having Claude explain existing code to help the human user understand the codebase), “refactoring” (using Claude to help restructure existing code), and “data science” (e.g. having Claude analyze datasets and make bar charts).

Below are the most common daily tasks. Most employees (55%) used Claude for debugging on a daily basis. 42% used Claude everyday for code understanding, and 37% used Claude everyday for implementing new features. The less-frequent tasks were high level design/planning (likely because these are tasks people tend to keep in human hands), as well as data science and front-end development (likely because they are overall less common tasks). This roughly aligns with the Claude Code usage data distribution reported in the “Claude Code usage trends” section.



Usage and productivity

Employees self-reported that 12 months ago, they used Claude in 28% of their daily work and got a +20% productivity boost from it, whereas now, they use Claude in 59% of their work and achieve +50% productivity gains from it on average. (This roughly corroborates the 67% increase in merged pull requests—i.e. successfully incorporated changes to code—per engineer per day we saw when we adopted Claude Code across our Engineering org.) The year-on-year comparison is quite dramatic—this suggests a more than 2x increase in both metrics in one year. Usage and productivity are also strongly correlated, and at the extreme end of the distribution, 14% of respondents are increasing their productivity by more than 100% by using Claude—these are our internal “power users.”

To caveat this finding (and other self-reported productivity findings below), productivity is difficult to precisely measure (see Appendix for more limitations). There is recent work from METR, an AI research nonprofit, showing that experienced developers working with AI on highly familiar codebases overestimated their productivity boost from AI. That being said, the factors that METR identified as contributing to lower productivity than expected (e.g. AI performing worse in large, complex environments, or where there’s a lot of tacit knowledge/context necessary) closely correspond to the types of tasks our employees said they *don’t* delegate to Claude (see AI delegation approaches, below). Our productivity gains, self-reported *across* tasks, might reflect employees developing strategic AI delegation skills—something not accounted

for in the METR study.

An interesting productivity pattern emerges when asking employees, for task categories where they currently use Claude, how it affects their overall time spent and work output volume in that task category. Across almost all task categories, we see a net decrease in time spent, and a larger net increase in output volume:

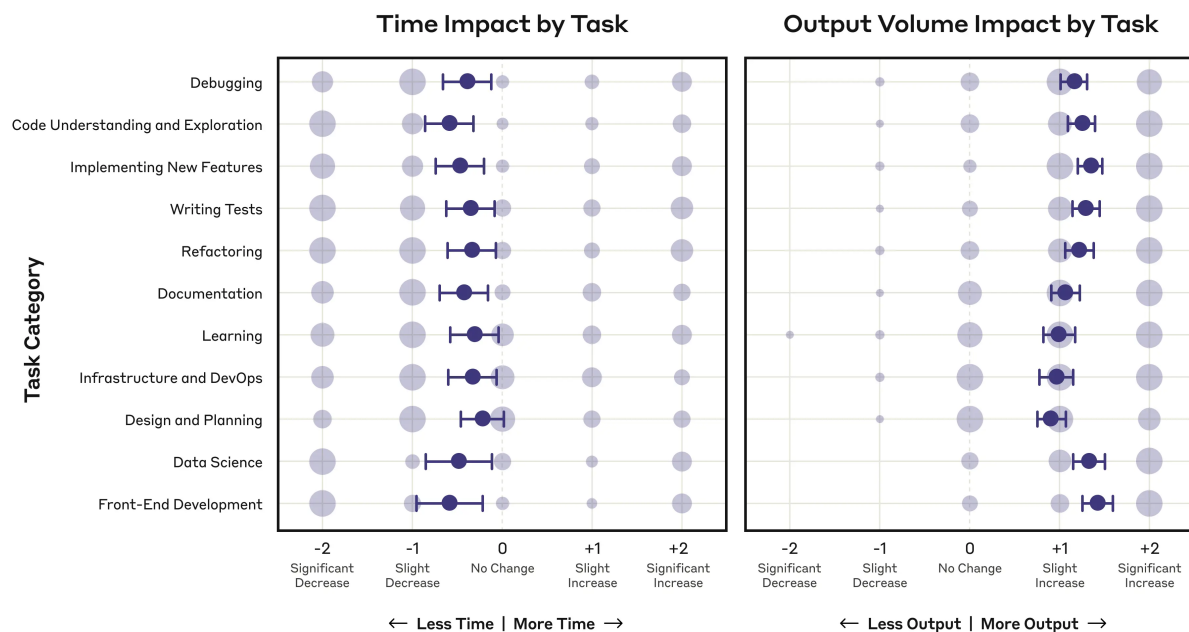


Figure 2: Impact on time spent (left panel) and output volume (right panel) by task (y-axis). The x-axis on each plot corresponds to either a self-reported decrease (negative values), increase (positive values) or no change (vertical dashed line) in time spent or output volume for categories of Claude-assisted tasks, compared to not using Claude. Error bars show 95% confidence intervals. Circle area is proportional to the number of responses at each rating point. Only respondents who reported using Claude for each task category are included.

However, when we dig deeper into the raw data, we see that the time saving responses cluster at opposite ends—some people spend significantly *more* time on tasks that are Claude-assisted.

Why is that? People generally explained that they had to do more debugging and cleanup of Claude’s code (e.g. “when I vibe code myself into a corner”), and shoulder more cognitive overhead for understanding Claude’s code since they didn’t write it themselves. Some mentioned spending more time on tasks in an enabling sense—one said that using Claude helps them “persist on tasks that I previously would’ve given up on immediately”; another said it helps them do

more thorough testing and also more learning and exploration in new codebases. It seems that generally, engineers experiencing time savings may be those who are scoping quickly-verifiable tasks for Claude, while those spending more time might be debugging AI-generated code or working in domains where Claude needs more guidance.

It is also not clear from our data where reported time savings are being reinvested—whether into additional engineering tasks, non-engineering tasks, interacting with Claude or reviewing its output, or activities outside of work. Our task categorization framework does not capture all the ways engineers might allocate their time. Additionally, the time savings may reflect perception biases in self-reporting. Further research is needed to disentangle these effects.

Output volume increases are more straightforward and substantial; there is a larger net increase across all task categories. This pattern makes sense when we consider that people are reporting on task categories (like “debugging” overall) rather than individual tasks—i.e. people can spend slightly less time on debugging as a category while producing much more debugging output overall. Productivity is very hard to measure directly, but this self-reported data suggests that AI enables increased productivity at Anthropic primarily through greater output volume.

Claude enabling new work

One thing we were curious about: Is Claude enabling qualitatively new kinds of work, or would Claude-assisted work have been done by employees eventually (albeit potentially at a slower rate)?

Employees estimated that 27% of their Claude-assisted work wouldn't have been done without it. Engineers cited using AI for scaling projects, nice-to-haves (e.g. interactive data dashboards), useful but tedious work like documentation and testing, and exploratory work that wouldn't be cost-effective manually. As one person explained, they can now fix more “papercuts” that previously damaged quality of life, such as refactoring badly-structured code, or building “small tools that help accomplish another task faster.” We looked for this in our usage data analysis as well, and found that 8.6% of Claude Code tasks involve ‘papercut fixes.’

Another researcher explained that they ran many versions of Claude simultaneously, all exploring different approaches to a problem:

People tend to think about super capable models as a single instance, like getting a faster car. But having a million horses... allows you to test a bunch of different ideas... It's exciting and more creative when you have that extra

| breadth to explore.

As we'll see in the following sections, this new work often involves engineers tackling tasks outside their core expertise.

How much work can be fully delegated to Claude?

Although engineers use Claude frequently, more than half said they can “fully delegate” only between 0-20% of their work to Claude. (It’s worth noting that there is variation in how respondents might interpret “fully delegate”—from tasks needing no verification at all to those that are reliable enough to require only light oversight.) When explaining why, engineers described working actively and iteratively with Claude, and validating its outputs—particularly for complex tasks or high-stakes areas where code quality standards are critical. This suggests that engineers tend to collaborate closely with Claude and check its work rather than handing off tasks without verification, and that they set a high bar for what counts as “fully delegated.”

Qualitative interviews

While these survey findings reveal significant productivity gains and changing work patterns, they raise questions about how engineers are actually experiencing these changes day-to-day. To understand the human dimension behind these metrics, we conducted in-depth interviews with 53 of the Anthropic engineers and researchers who responded to the survey, to get more insight into how they’re thinking and feeling about these changes in the workplace.

AI delegation approaches

Engineers and researchers are developing a variety of strategies for productively leveraging Claude in their workflow. People generally delegate tasks that are:

Outside the user's context and low complexity:

"I use Claude for things where I have low context, but think that the overall complexity is also low."

"The majority of the infra[structure] problems I have are not difficult and can be handled by Claude... I don't know Git or Linux very well... Claude does a good job covering for my lack of experience in these areas."

Easily verifiable:

"It's absolutely *amazing* for everything where validation effort isn't large in comparison to creation effort."

Well-defined or self-contained:

"If a subcomponent of the project is sufficiently decoupled from the rest, I'll get Claude to take a stab."

Code quality isn't critical:

"If it's throwaway debug[ging] or research code, it goes straight to Claude. If it's conceptually difficult or needs some very specific type of debug injection, or a design problem, I do it myself."

Repetitive or boring:

"The more excited I am to do the task, the more likely I am to not use Claude. Whereas if I'm feeling a lot of resistance... I often find it easier to start a conversation with Claude about the task."

In our survey, on average people said that 44% of Claude-assisted work consisted of tasks they wouldn't have enjoyed doing themselves.

Faster to prompt than execute:

"[For] a task that I anticipate will take me less than 10 minutes... I'm probably not going to bother using Claude."

"The cold start problem is probably the biggest blocker right now. And by cold start, I mean there is a lot of intrinsic information that I just have about how my team's code base works that Claude will not have by default... I could spend time trying to iterate on the perfect prompt [but] I'm just going to go and do it myself."

These factors mentioned by our employees in their decisions about delegation were similar to those found to explain AI-related productivity slowdowns (such as high developer familiarity with codebase, large and complex repositories) in an external study from METR. The convergence on these delegation criteria

across our interviews suggests that appropriate task choice is an important factor in AI productivity gains (which should be carefully controlled for in future productivity studies).

Trust but verify

Many users described a progression in their Claude usage that involved delegating increasingly complex tasks over time: “At first I used AI tools with basic questions about Rust programming language... Lately, I've been using Claude Code for all my coding.”

One engineer likened the trust progression to adopting other technologies, like Google Maps:

In the beginning I would use [Google Maps] only for routes I didn't know... This is like me using Claude to write SQL that I didn't know, but not asking it to write Python that I did. Then I started using Google Maps on routes that I mostly knew, but maybe I didn't know the last mile... Today I use Google Maps all the time, even for my daily commute. If it says to take a different way I do, and just trust that it considered all options... I use Claude Code in a similar way today.

Engineers are split on whether to use Claude within or outside their expertise. Some use it for “peripheral” domains to save implementation time; others prefer familiar territory where they can verify outputs (“I use Claude in such a way where I still have full understanding of what it's doing”). A security engineer highlighted the importance of experience when Claude proposed a solution that was “really smart in the dangerous way, the kind of thing a very talented junior engineer might propose”. That is, it was something that could only be recognised as problematic by users with judgment and experience.

Other engineers use Claude for both types of tasks, either in an experimental way (“I basically always use Claude to take a first crack at any coding problem”), or by adapting their approach depending on their level of expertise in the task:

I use the tools for both things that are core to my expertise (as an accelerant, where I know what to expect and can guide the agent effectively), and for things that are slightly outside my area of expertise, where I know roughly what to expect but that Claude is able to fill in the gaps in my memory or familiarity with specific definitions.

If it's something that I am particularly versed about, I will be more assertive and tell Claude what it needs to track down. If it's something I'm not sure about I often ask it to be the expert and give me options and insights on things I should consider and research.

What tasks do people keep for themselves?

People consistently said they didn't use Claude for tasks involving high-level or strategic thinking, or for design decisions that require organizational context or "taste." One engineer explained: "I usually keep the high-level thinking and design. I delegate anything I can from new feature development to debugging." This is reflected in our survey data, which showed the least productivity gains for design and planning tasks (Figure 2). Many people described delegation boundaries as a "moving target," though, regularly renegotiated as models improve (below, the Claude Code usage data shows relatively more coding design/planning usage now than six months ago).

Skill transformations

New capabilities...

The survey finding that 27% of Claude-assisted work wouldn't have been done otherwise reflects a broader pattern: engineers using AI to work outside their core expertise. Many employees report completing work previously outside their expertise—backend engineers building UIs; researchers creating visualizations. One backend engineer described building a complex UI by iterating with Claude: "It did a way better job than I ever would've. I would not have been able to do it, definitely not on time... [The designers] were like 'wait, you did this?' I said 'No, Claude did this - I just prompted it.'"

Engineers report "becoming more full-stack... I can very capably work on front-end, or transactional databases, or API code, where previously I would've been scared to touch stuff I'm less of an expert on." This capability expansion enables tighter feedback loops and faster learning—one engineer said that a "couple week process" of building, scheduling meetings, and iterating could become "a couple hour working session" with colleagues present for live feedback.

In general, people were enthused by their new ability to prototype quickly, parallelize work, reduce toil, and generally raise their level of ambition. One senior engineer told us, "The tools are definitely making junior engineers more productive and more bold with the types of projects they will take on." Some also said that the reduced "activation energy" of using Claude enabled them to defeat procrastination more easily, "dramatically decreas[ing] the energy required for me to want to start tackling a problem and therefore I'm willing to tackle so many additional things."

...and less hands-on practice

At the same time, some were worried about "skills atrophying as [they] delegate more", and losing the incidental (or "collateral") learning that happens during manual problem-solving:

If you were to go out and debug a hard issue yourself, you're going to spend time reading docs and code that isn't directly useful for solving your problem—but this entire time you're building a model of how the system works. There's a lot less of that going on because Claude can just get you to the problem right away.

I used to explore every config to understand what the tool can do but now I rely on AI to tell me how to use new tools and so I lack the expertise. In conversations with other teammates I can instantly recall things vs now I have to ask AI.

Using Claude has the potential to skip the part where I learn how to perform a task by solving an easy instance, and then struggle to solve a more complicated instance later.

One senior engineer said they'd be more worried about their skills if they were more junior:

I'm primarily using AI in cases where I know what the answer should be or should look like. I developed that ability by doing SWE 'the hard way'... But if I were [earlier in my career], I would think it would take a lot of deliberate effort to continue growing my own abilities rather than blindly accepting the model output.

One reason that the atrophy of coding skills is concerning is the “paradox of supervision”—as mentioned above, effectively using Claude requires supervision, and supervising Claude requires the very coding skills that may atrophy from AI overuse. One person said:

Honestly, I worry much more about the oversight and supervision problem than I do about my skill set specifically... having my skills atrophy or fail to develop is primarily gonna be problematic with respect to my ability to safely use AI for the tasks that I care about versus my ability to independently do those tasks.

To combat this, some engineers deliberately practice without AI: "Every once in a while, even if I know that Claude can nail a problem, I will not ask it to. It helps me keep myself sharp."

Will we still need those hands-on coding skills?

Perhaps software engineering is moving to higher levels of abstraction, which it has done in the past. Early programmers worked much closer to the machine—

manually managing memory, writing in assembly language, or even toggling physical switches to input instructions. Over time, higher-level, more human-readable languages emerged that automatically handled complex, low-level operations. Perhaps, in particular with the rise of “vibe coding”, we’re now moving to English as a programming language. One of our staff suggested that aspiring engineers “get good at having AIs [write code], and focus on learning higher level concepts and patterns.”

A few employees said they felt that this shift empowers them to think at a higher level—“about the end product and the end user” rather than just the code. One person described the current shift by comparing it to previously having to learn linked-lists in computer science—fundamental structures that higher-level programming languages now handle automatically. “I’m very glad I knew how to do that... [but] doing those low level operations isn’t particularly important emotionally. I would rather care about what the code allows me to do.” Another engineer made a similar comparison, but noted that abstraction comes at a cost—with the move to higher-level languages, most engineers lost a deep understanding of memory handling.

Continuing to develop skills in an area can lead to better supervision of Claude and more efficient work (“I notice that when it’s something I’m familiar with, it’s often faster for me to do it”). But engineers are divided on whether this matters. Some remain sanguine:

I don't worry too much about skill erosion. The AI still makes me think through problems carefully and helps me learn new approaches. If anything, being able to explore and test ideas more quickly has accelerated my learning in some areas.

Another was more pragmatic: “I am for sure atrophying in my skills as a software engineer... But those skills could come back if they ever needed to, and I just don't need them anymore!” One noted they only lost less-important skills like making charts, and “the kind of code that’s critical I can still write very well.”

Perhaps most interestingly, one engineer challenged the premise: “The ‘getting rusty’ framing relies on an assumption that coding will someday go back to the way it was pre-Claude 3.5. And I don't think it will.”

The craft and meaning of software engineering

Engineers diverge sharply on whether they miss hands-on coding. Some feel genuine loss—“It’s the end of an era for me - I’ve been programming for 25 years, and feeling competent in that skill set is a core part of my professional satisfaction.” Others worry about not enjoying the new nature of the work: “Spending your day prompting Claude is not very fun or fulfilling. It’s much more fun and fulfilling to put on some music and get in the zone and implement

something yourself.”

Some directly addressed the trade-off and accepted it: “There are certainly some parts of [writing code] that I miss - getting into a zen flow state when refactoring code, but overall I'm so much more productive now that I'll gladly give that up.”

One person said that iterating with Claude has been *more* fun, because they can be more picky with their feedback than with humans. Others are more interested in outcomes. One engineer said:

I expected that by this point I would feel scared or bored... however I don't really feel either of those things. Instead I feel quite excited that I can do significantly more. I thought that I really enjoyed writing code, and instead I actually just enjoy what I get out of writing code.

Whether people embrace AI assistance or mourn the loss of hands-on coding seems to depend on what aspects of software engineering they find most meaningful.

Changing social dynamics in the workplace

One of the more prominent themes was that Claude has become the first stop for questions that once went to colleagues. “I ask way more questions [now] in general, but like 80-90% of them go to Claude,” one employee noted. This creates a filtering mechanism where Claude handles routine inquiries, leaving colleagues to address more complex, strategic, or context-heavy issues that exceed AI capabilities (“It has reduced my dependence on [my team] by 80%, [but] the last 20% is crucial and I go and talk to them”). People also “bounce ideas off” Claude, similar to interactions with human collaborators.

About half reported unchanged team collaboration patterns. One engineer said that he was still meeting with people, sharing context, and choosing directions, and that he thought that in the near future there'd still be a lot of collaboration, but “instead of doing your standard focus work, you'll be talking to a lot of Claudes.”

However, others described experiencing less interaction with colleagues (“I work way more with Claude than with any of my colleagues.”) Some appreciate the reduced social friction (“I don't feel bad about taking my colleague's time”). Others resist the change (“I actually don't love that the common response is ‘have you asked Claude?’ I really enjoy working with people in person and highly value that”) or miss the older way of working: “I like working with people and it is sad that I ‘need’ them less now.” Several pointed out the impact on traditional mentorship dynamics, because “Claude can provide a lot of coaching to junior staff” instead of senior engineers. One senior engineer said:

It's been sad that more junior people don't come to me with questions as often, though they definitely get their questions answered more effectively and learn faster.

Career uncertainty and adaptation

Many engineers describe their role shifting from writing code to managing AIs. Engineers increasingly see themselves as “manager[s] of AI agents”—some already “constantly have at least a few [Claude] instances running.” One person estimated their work has shifted “70%+ to being a code reviewer/reviser rather than a net-new code writer” and another saw “taking accountability for the work of 1, 5, or 100 Claudes” as part of their future role.

In the longer term, career uncertainty is widespread. Engineers saw these changes as harbingers of broader industry transformation, and many said that it was “hard to say” what their careers might look like a few years down the line. Some expressed a conflict between short-term optimism and long-term uncertainty. “I feel optimistic in the short term but in the long term I think AI will end up doing everything and make me and many others irrelevant,” one stated. Others put a finer point on it: “It kind of feels like I'm coming to work every day to put myself out of a job.”

Some engineers were more optimistic. One said, “I fear for the junior devs, but I also appreciate that junior devs are maybe the thirstiest for new technology. I feel generally very optimistic about the trajectory of the profession.” They argued that, while there’s a potential risk of inexperienced engineers shipping problematic code, the combination of better AI guardrails, more built-in educational resources, and natural learning from mistakes will help the field adapt over time.

We asked how people envision their future roles and whether they have any adaptation strategies. Some mentioned plans to specialize further (“developing the skill to meaningfully review AI’s work will take longer and require more specialization”), some anticipated focusing on more interpersonal and strategic work in the future (“we will spend more time finding consensus and let the AIs spend more time on the implementation”). One said they use Claude specifically for career development, getting feedback from it on work and leadership skills (“The rate at which I can learn things or even just be effective without fully learning things just completely changed. I almost feel like the ceiling just shattered for me”).

Overall, many acknowledge deep uncertainty: “I have very low confidence in what specific skills I think will be useful in the future.” A team lead said: “Nobody knows what's going to happen... the important thing is to just be really

adaptable.”

Claude Code usage trends

The survey and interview data show that increased Claude usage is helping people work faster and take on new types of work, though this comes with tensions around AI delegation and skill development. Still, self-reported data only tells part of the story. To complement this, we also analyzed actual Claude usage data across Anthropic teams. Because survey respondents reported Claude Code as the majority of their usage, we used our [privacy-preserving analysis tool](#) to analyze 200,000 internal transcripts from Claude Code from February and August 2025.

Tackling harder problems with less oversight

Claude Code usage has shifted toward more difficult and autonomous coding tasks over the last six months: (Figure 3):

- **Employees are tackling increasingly complex tasks with Claude Code.** We estimated task complexity of each transcript on a 1-5 scale where 1 corresponds to “basic edits” and 5 is “expert-level tasks requiring weeks/months of human expert work”. Task complexity increased from 3.2 to 3.8 on average. To illustrate the difference between the scores: tasks averaging 3.2 included “Troubleshoot Python module import errors” while tasks averaging 3.8 included “Implement and optimize caching systems.”
- **The maximum number of consecutive tool calls Claude Code makes per transcript increased by 116%.** Tool calls correspond to actions Claude takes using external tools like making edits to files or running commands. Claude now chains together 21.2 independent tool calls without need for human intervention versus 9.8 tool calls from six months ago.
- **The number of human turns decreased by 33%.** The average number of human turns decreased from 6.2 to 4.1 per transcript, suggesting that less human input is necessary to accomplish a given task now compared to six months ago.

These usage data corroborate the survey data: engineers delegate increasingly complex work to Claude and Claude requires less oversight. It seems plausible that this is driving the observed productivity gains.

Distribution of tasks

We classified Claude Code transcripts into one or more types of coding tasks,

studying how the uses for different tasks have evolved over the last six months:

The overall task frequency distribution estimated from usage data roughly aligns with the self-reported task frequency distribution. The most striking change between February and August 2025 is that there now are proportionately many more transcripts using Claude to implement new features (14.3% → 36.9%) and do code design or planning (1.0% → 9.9%). This shift in the relative distribution of Claude Code tasks may suggest that Claude has become better at these more complex tasks, though it could also reflect changes in how teams adopt Claude Code for different workflows rather than increases in absolute work volume (see Appendix for more limitations).

Fixing papercuts

We found from the survey that engineers now spend more time making small quality-of-life improvements; in line with this, 8.6% of current Claude Code tasks are classified as “papercut fixes”. These include larger tasks such as creating performance visualization tools and refactoring code for maintainability, as well as smaller tasks like creating terminal shortcuts. This may contribute to engineers’ reported productivity gains (addressing previously neglected quality-of-life improvements may lead to more efficiency over time) and potentially reducing friction and frustration in daily work.

Task variation across teams

To study how tasks currently vary across teams, we refined our classification approach to assign each August transcript to a single primary coding task, and split the data by internal teams (y-axis). The stacked bar chart shows the breakdown of coding tasks for each team:

The "All Teams" bar shows the overall distribution, with the most common tasks being building new features, debugging, and code understanding. This provides a baseline for team-specific comparisons.

Notable team-specific patterns:

1. The **Pre-training** team (who help to train Claude) often uses Claude Code for building new features (54.6%), much of which is running extra experiments.
2. The **Alignment & Safety** and **Post-training** teams do the most front-end development (7.5% and 7.4%) with Claude Code, often for creating data visualizations.
3. The **Security** team often uses Claude Code for code understanding (48.9%), specifically analyzing and understanding the security implications of different parts of the codebase.
4. **Non-technical** employees often use Claude Code for debugging (51.5%), such as troubleshooting network issues or Git operations, as well as for data science

(12.7%); Claude appears to be valuable for bridging gaps in technical knowledge.

Many of these team-specific patterns demonstrate the same capability expansion we observed in our survey and interviews: enabling new kinds of work that those on the team either wouldn't have the time or the skillset to do otherwise. For example, the pretraining team ran lots of additional experiments and non-technical employees were able to fix errors in code. And whereas the data suggests that teams do use Claude for their core tasks (for instance, the Infrastructure team most commonly uses Claude Code for infrastructure and DevOps work), Claude often also augments their core tasks (for instance, researchers use Claude for front-end development to better visualize their data). This suggests that Claude is enabling everyone to become more full-stack in their work.

Looking forward

Anthropic employees have greatly increased their use of Claude over the past year, using it to not only accelerate existing work but to learn new codebases, reduce toil, expand into new domains, and tackle previously neglected improvements. As Claude becomes more autonomous and capable, engineers are discovering new ways to use AI delegation while also figuring out what skills they'll need in the future. These shifts bring clear productivity and learning benefits alongside genuine uncertainty about the longer-term trajectory of software engineering work. Will AI resemble past software engineering transitions—from lower- to higher-level programming languages, or from individual contributor to manager, as several engineers suggested? Or will it go further?

It's still early days—Anthropic has many early adopters internally, the landscape is rapidly changing, and our findings likely don't generalize to other organizations or contexts right now (see Appendix for more limitations). This research reflects that uncertainty: the findings are nuanced, with no single consensus or clear directives emerging. But it does raise questions about how we can thoughtfully and effectively navigate these changes.

To follow up on this initial work, we're taking several steps. We're talking to Anthropic engineers, researchers, and leadership to address the opportunities and challenges raised. This includes examining how we bring teams together and collaborate with each other, how we support professional development, and/or how we establish best practices for AI-augmented work (e.g. guided by our AI fluency framework). We're also expanding this research beyond engineers to understand how AI transformation affects roles across the organization and

supporting external organizations such as CodePath as they adapt computer science curricula for an AI-assisted future. Looking ahead, we're also considering structural approaches that may become increasingly relevant as AI capabilities advance, like new pathways for role evolution or reskilling within the organization.

We expect to share more concrete plans in 2026 as our thinking matures. Anthropic is a laboratory for responsible workplace transition; we want to not just study how AI transforms work, but also experiment with how to navigate that transformation thoughtfully, starting with ourselves first.

